IMD0904 –Plataforma de Hardware para Internet das Coisas

Prof. Daniel Lopes Martins

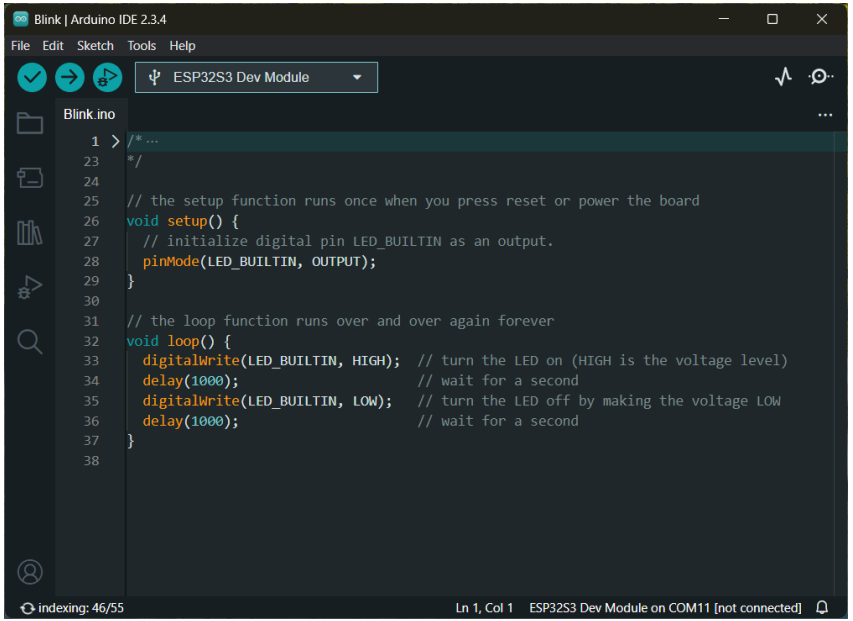
UFRN | metrópole DIGITAL

AULA 05

Programação em C para microcontroladores ESP32

Introdução

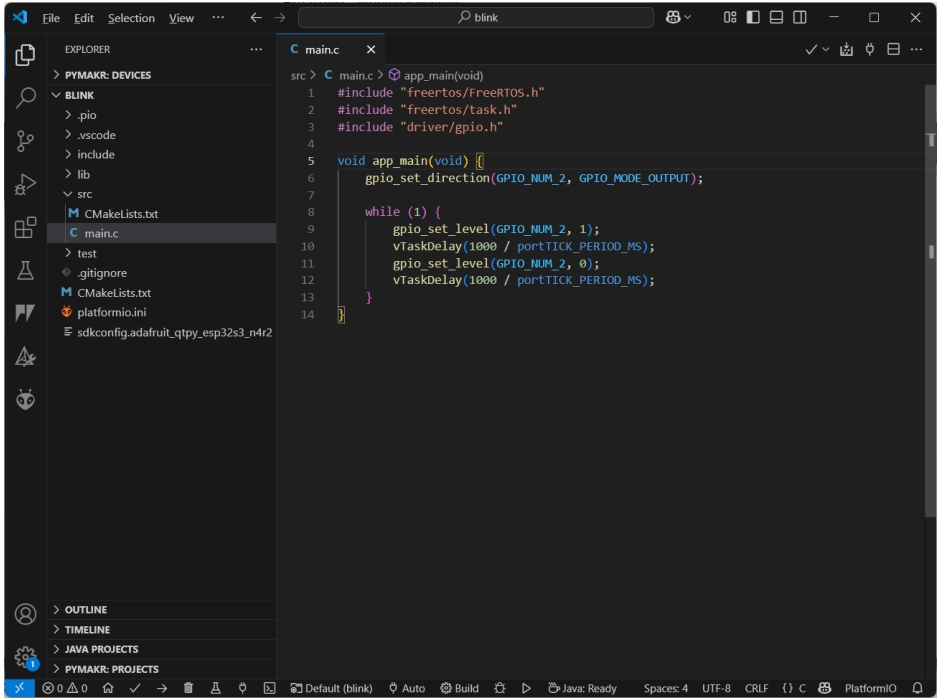
- Apesar da linguagem base utilizada para programação no arduino IDE ser C/C++, ela possui diferenças importantes:
  - Possui um nível de abstração alto.
  - Se torna muito útil para iniciantes ou projetos simples.
  - Também é muito boa para desenvolvimento de protótipos e possui vasta documentação.
  - Usar arduino IDE com ESP32 também permite fácil integração com bibliotecas arduino.



[1]

Introdução

- C/C++ da IDF/plataformio:
  - É ideal para projetos profissionais.
  - Permite uso avançado do FreeRTOS, WiFi, BLE, OTA, segurança entre outras funcionalidades.
  - Mais estável e performático para aplicações robustas.
  - Controle detalhado de clock, memórias e periféricos.



[1]

Comparativo

Aspecto	Estilo Arduino	ESP-IDF
Nível de abstração	Alto (fácil e pronto pra usar)	Baixo (mais próximo do hardware)
Linguagem base	C++ com funções simplificadas	C com estrutura modular e explicita e suporte a C++
Estrutura do programa	setup() e loop()	app_main()
Bibliotecas	Simplificadas e unificadas	Drivers nativos por componente
Controle de hardware	Abstraído por funções fáceis	Controle direto via registradores e APIs
RTOS	Existe, mas escondido	FreeRTOS explícito e configurável
Gerenciamento de tarefas	Implícito via loop()	Explicito via xTaskCreate, sem loop()
Build System	Simplificado	Cmake ou Make
Performance e eficiência	Boa, mas limitada	Máxima flexibilidade e otimização

Diferenças em código

- Estrutura do programa
  - O estilo Arduino usa setup() e o loop implícito e automático na função loop().
  - O ESP-IDF usa app_main() e loop manual com while(1) ou xTaskCreate().

```
void setup() {
  // Inicialização
}

void loop() {
  // Repetição contínua
}
```

```
void app_main() {
  // Código principal
  while (1) {
    // Repetição manual
  }
}
```

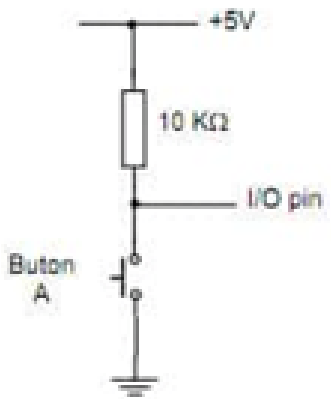
Diferenças em código

- Acesso ao GPIO e configurações
  - O estilo Arduino usa:
    - pinMode(pin, mode) – Função que define se o pino deve ser utilizado como entrada ou saída no microcontrolador. O mode pode ser: INPUT (entrada), INPUT_PULLUP (entrada com resistor de pullup), INPUT_PULLDOWN (entrada com resistor de pulldown) e OUTPUT (saída).
    - digitalWrite(pin, HIGH/LOW) – Função que escreve/altera o estado do pino (nível alto ou baixo). O HIGH coloca o pino no estado de nível alto (no caso do ESP32, o nível alto equivale a tensão de 3.3V) e o LOW coloca o pino no estado de nível baixo (0V).
    - digitalRead(pin) – Função que retorna o estado do pino. Se for aplicado 3.3V ou 0V ao pino, a função retorna um inteiro 1 ou 0 respectivamente.

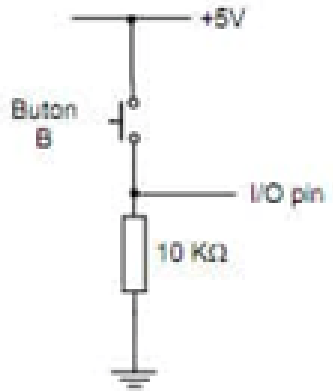
```
pinMode(2, OUTPUT);
digitalWrite(2, HIGH);
```

Diferenças em código

- Acesso ao GPIO e configurações
  - Resistor de pullup e pulldown:
    - Quando você tem um pino de entrada e não conecta nada a ele, ele fica em alta impedância (não está com referência nem 0 V nem VCC). Nesse estado, ele pode captar ruído eletromagnético e mudar aleatoriamente entre 0 e 1, causando comportamentos imprevisíveis.
    - Um resistor de pull-up ou pull-down são componentes usados nesses circuitos digitais para definir um nível lógico padrão (HIGH ou LOW) na entrada que ficaria flutuando (indefinida).



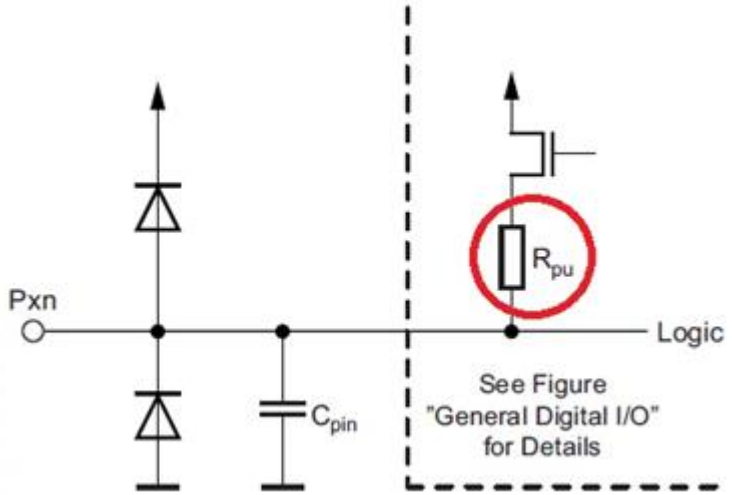
Pull-Up



Pull-Down [2]

Diferenças em código

- Acesso ao GPIO e configurações
  - Resistor de pullup e pulldown:
    - No caso, os microcontroladores podem ter internamente tais resistores e eles são ativados por programação. Usando a função pinMode com o parâmetro INPUT_PULLUP ou INPUT_PULLDOWN ativa respectivamente o resistor de pullup e pulldown.



[3]

Diferenças em código

- Acesso ao GPIO e configurações
  - ESP-IDF usa:
    - gpio_set_direction(GPIO_NUM_x, GPIO_MODE_xxx) – Função equivalente a pinMode.
      - GPIO_MODE pode ser:
        - DISABLE – Desativa o pino.
        - INPUT – Pino como entrada.
        - OUTPUT – Pino como saída.
        - OUTPUT_OD – Saída com Open-Drain.
        - INPUT_OUTPUT – Habilita entrada e saída ao mesmo tempo (útil com barramentos bidirecionais e configuração Open-Drain).
        - INPUT_OUTPUT_OD – Entrada e saída com Open-Drain.
    - gpio_set_level(GPIO_NUM_x, 1/0) – Função que define o valor do pino, onde 1 é nível alto (no caso do ESP32, 3,3V). Equivale ao digitalWrite do Arduino IDE.

Diferenças em código

- Acesso ao GPIO e configurações
  - ESP-IDF usa:
    - gpio_set_pull_mode(GPIO_NUM_x, GPIO_MODE)
      - O MODE pode ser:
        - GPIO_FLOATING – Nenhum resistor ativo no pino.
        - GPIO_PULLUP_ONLY – Ativa o pull-up interno.
        - GPIO_PULLDOWN_ONLY – Ativa o pull-down interno.
        - GPIO_PULLUP_PULLDOWN – Ativa ambos os resistores (quase nunca usado).
    - gpio_pullup_en(GPIO_NUM_x) – Ativa o pull-up no pino desejado.
    - gpio_pullup_dis(GPIO_NUM_x) – Desativa o pull-up no pino desejado.
    - gpio_pulldown_en(GPIO_NUM_x) – Ativa o pull-down.
    - gpio_pulldown_dis(GPIO_NUM_x) – Desativa o pull-down.

```
gpio_set_direction(GPIO_NUM_2, GPIO_MODE_OUTPUT);
gpio_set_level(GPIO_NUM_2, 1);
```

Diferenças em código

- Acesso ao GPIO e configurações
  - ESP-IDF usa:
    - Para configuração mais completa, gpio_config() com o struct gpio_config_t.

```
// Configuração dos pinos de entrada com pull-up
   gpio_config_t io_conf_in;
   io_conf_in.intr_type = GPIO_INTR_DISABLE;                    // Sem interrupção
   io_conf_in.mode = GPIO_MODE_INPUT;                           // Modo entrada
   io_conf_in.pin_bit_mask = (1ULL << GPIO_NUM_2) | (1ULL << GPIO_NUM_4); // Máscara
dos pinos 2 e 4
   io_conf_in.pull_down_en = GPIO_PULLDOWN_DISABLE;
   io_conf_in.pull_up_en = GPIO_PULLUP_ENABLE;
   gpio_config(&io_conf_in);                                    // Aplica a configuração
```

Diferenças em código

- Atrasos
  - Estilo Arduino usa delay(ms) ou delayMicroseconds(us)
    - Bloqueia totalmente o código enquanto o tempo não passa (espera ocupada).
    - Útil para coisas simples.
    - Nenhuma outra instrução é executada, mesmo funções de hardware como timers ou comunicação serial podem ficar paradas.
    - Não permite fazer multitarefas ou usar o FreeRTOS de forma eficiente.
  - ESP-IDF usa vTaskDelay(ms / portTICK_PERIOD_MS) ou (pdMS_TO_TICKS(ms))
    - Funciona dentro de tarefas do FreeRTOS.
    - Não bloqueia o processador, apenas a tarefa atual.
    - Recebe ticks do RTOS, não milissegundos diretamente. A função pdMS_TO_TICKS(ms) converte isso.

Diferenças em código

- Atrasos
  - ESP-IDF usa vTaskDelay(ms / portTICK_PERIOD_MS) ou (pdMS_TO_TICKS(ms))
    - Ticks é a menor unidade de tempo do RTOS para gerenciar as tarefas.
    - Configurada no arquivo FreeRTOSConfig.h
      - #define configTICK_RATE_HZ 1000 // 1000 Hz → 1 ms por tick
    - Usar a função pdMS arredonda a conversão para baixo sendo mais segura para usar no FreeRTOS.
    - ms / portTICK_PERIOD_MS usa truncamento simples, o que pode levar a perda de precisão para valores de ms não múltiplos do portTICK_PERIOD_MS.

```
delay(1000);
```

```
vTaskDelay(1000 / portTICK_PERIOD_MS);
vTaskDelay(pdMS_TO_TICKS(1000));
```

Diferenças em código

- Atrasos
  - E se quisesse uma espera não ocupada?
    - Combinação de um contador (de tempo ou ticks) + um condicional.
    - Esse contador de tempo nada mais é que um registrador que conta quanto tempo se passou desde o início do microcontrolador (boot).
    - No Arduino (Atmel) esse contador é um registrador de 32 bits e é incrementado por uma interrupção que é acionada pelo timer de 8 bits do MCU (timer0).
    - O tempo de estouro desse contador é (funções do estilo Arduino):
      - milis() – aproximadamente 49,7 dias.
      - micros() – Aproximadamente 71,5 minutos (resolução de 4 µs).
    - O ESP32 utiliza um timer de alta resolução (64 bits) incrementado a cada ciclo do cristal. O tempo de estouro em microssegundos desse timer é de aproximadamente 292 anos.

Diferenças em código

- Atrasos
  - No ESP-IDF existe a função esp_timer_get_time() que devolve os µs e a função xTaskGetTickCount() devolve os ticks do FreeRTOS (em ms).
  - A principal diferença entre elas:
    - esp_timer_get_time():
      - Retorna em microssegundos desde o boot do ESP32.
      - Usa o timer de hardware de 64 bits.
      - Precisão de 1 µs.
      - Contador independente.
    - xTaskGetTickCount():
      - Retorna o número de ticks do FreeRTOS desde o boot.
      - Resolução de 1 tick = 1/configTick_Rate_HZ segundos (normalmente 1 ms) e o contador é de 32 bits (estoura em aproximadamente 49,7 dias).
      - Depende do scheduler do FreeRTOS, sendo incrementado apenas quando o RTOS atualiza seus contadores.

Diferenças em código

- Atrasos
  - Exemplo de espera não ocupada em estilo Arduino:

```
// Declaração fora do laço inicialmente
unsigned long previousMillis = 0;
const long interval = 1000; // 1 segundo
// Dentro do laço
unsigned long currentMillis = millis();                  // tempo atual em milisegundos
if (currentMillis - previousMillis >= interval) {        // interval é o tanto de tempo a esperar
    previousMillis = currentMillis;                      // atualiza marcador
    // ação a ser executada
}
// faz outras tarefas sem bloqueio
```

Diferenças em código

- Atrasos
  - Exemplo de espera não ocupada no ESP-IDF utilizando esp_timer_get_time():

```
// Declaração fora do laço inicialmente
uint64_t previousTime = 0;
const uint64_t interval = 1000000; // 1 segundo em microsegundos
// Dentro do laço
uint64_t currentTime = esp_timer_get_time();
if (currentTime - previousTime >= interval) {
    previousTime = currentTime;
    // ação a ser executada
}
// Executa outras tarefas sem bloqueio
```

Diferenças em código

- Atrasos
  - Exemplo de espera não ocupada no ESP-IDF utilizando xTaskGetTickCount():

```
// Declaração fora do laço inicialmente
TickType_t previousTick = xTaskGetTickCount();
const TickType_t interval = pdMS_TO_TICKS(1000);
// Dentro do laço
TickType_t currentTick = xTaskGetTickCount();
if (currentTick - previousTick >= interval) {
    previousTick = currentTick;
    // ação a ser executada
}
// Executa outras tarefas sem bloqueio
```

Diferenças em código

- Leitura de dados analógicos
  - Estilo Arduino:
    - O Arduino possui um conversor analógico digital (ADC), mas possui 6 canais.
    - Isso significa que internamente ele possui um mux que multiplexa 6 pinos externos (A0 a A5) para esse ADC.
    - O conversor é de 10 bits (resolução de 1024 níveis – 0 a 1023).
    - Tensão de trabalho: 0 – 5 V.
    - Taxa de conversão de aproximadamente 10KS/s (Amostras/s).
    - No estilo Arduino usa-se a função analogRead(pin) para ler o dado analógico de um pino (canal).

```
int valor = analogRead(34);
```

Diferenças em código

- Leitura de dados analógicos
  - ESP-IDF:
    - O ESP possui 2 conversores A/D, sendo que o ADC1 tem 8 canais e o ADC2 tem 10 canais (mesma lógica de multiplexação do Arduino).
    - Os conversores são de 12 bits, com resolução de 4096 níveis (0 a 4095).
    - Tensão de Trabalho: 0 - 1,1 V (ou até 3,9 V com atenuadores).
    - Taxa de conversão na leitura simples é de aproximadamente 200 KS/s (baixo consumo) até 2 MS/s na leitura contínua.
    - No ESP-IDF usa-se a função adcY_get_raw(ADCY_CHANNEL_x) para ler o dado analógico de um pino (canal x) utilizando o adc Y (1 ou 2).
    - O ADC do ESP32 também possui uma configuração de atenuação que permite alterar a faixa de leitura de tensão na entrada.

Diferenças em código

- Tabela de atenuações do ADC

Atenuação	Faixa de tensão	Descrição
ADC_ATTEN_DB_0	0,0V – 1,1V	Sem atenuação (1,1V máxima)
ADC_ATTEN_DB_2_5	0,0V – 1,5V	Atenuação de 2,5 dB
ADC_ATTEN_DB_6	0,0V – 2,2V	Atenuação de 6 dB
ADC_ATTEN_DB_11	0,0V – 3,9V	Atenuação de 11 dB

- Cuidados com o ADC2
  - Quando utilizando o WiFi, pode haver problemas com os canais do ADC2, pois compartilham o mesmo hardware.
  - Recomenda-se utilizar apenas o ADC1 quando utilizando o WiFi.

Diferenças em código

- Pinos conectados aos ADCs no ESP32

Canal (ADC1)	GPIO	Canal (ADC2)	GPIO
ADC1_CH0	GPIO36	ADC2_CH0	GPIO4
ADC1_CH1	GPIO37	ADC2_CH1	GPIO0
ADC1_CH2	GPIO38	ADC2_CH2	GPIO2
ADC1_CH3	GPIO39	ADC2_CH3	GPIO15
ADC1_CH4	GPIO32	ADC2_CH4	GPIO13
ADC1_CH5	GPIO33	ADC2_CH5	GPIO12
ADC1_CH6	GPIO34	ADC2_CH6	GPIO14
ADC1_CH7	GPIO35	ADC2_CH7	GPIO27
		ADC2_CH8	GPIO25
		ADC2_CH9	GPIO26

Diferenças em código

- Leitura de dados analógicos
  - Funções utilizadas na configuração e leitura dos ADCs do ESP32 no ESP-IDF:
    - adcY_config_width(ADC_WIDTH_BIT_BB) – define a resolução do ADC onde BB pode variar de 9 a 12 bits e Y é o ADC (1 ou 2).
    - adcY_config_channel_atten(ADC1_CHANNEL_C, ADC_ATTEN_DB_DD) – define a atenuação DD (como visto na tabela) para um canal C do ADC para medir tensões maiores.
    - adcY_get_raw(ADCY_CHANNEL_C) – retorna a leitura do adc Y no canal C.
  - Exemplo de leitura de um ADC do esp32:

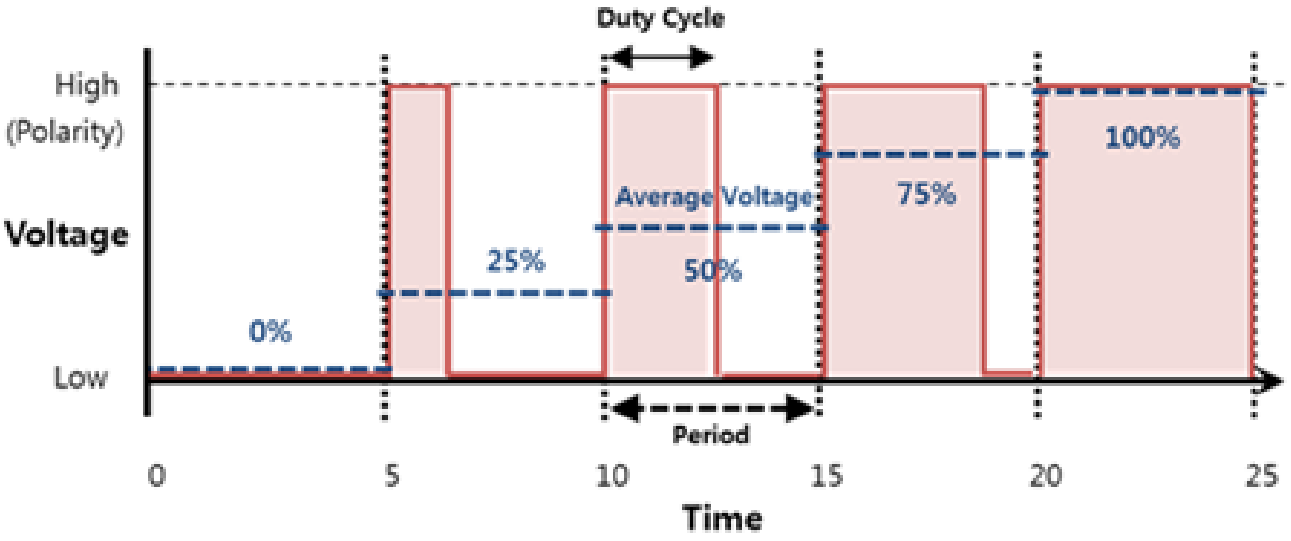
```
adc1_config_width(ADC_WIDTH_BIT_12);
adc1_config_channel_atten(ADC1_CHANNEL_6,
ADC_ATTEN_DB_11);
int valor = adc1_get_raw(ADC1_CHANNEL_6);
```

Diferenças em código

- Escrita de dados analógicos
  - Pulse Width Modulation (PWM)
    - A modulação por largura de pulso é uma técnica que nos permite simular um sinal analógico usando um sinal digital. Em vez de variar a tensão de forma continua (como com o uso de um potenciômetro), o PWM liga e desliga a energia em uma velocidade alta.
    - Composto por 2 elemento principais:
      - Ciclo de trabalho (duty cycle): porcentagem do tempo que o sinal fica ligado em um ciclo.
      - Frequência: Velocidade com que esses ciclos se repetem. Para controlar um motor, a frequência tem que ser alta o suficiente para que ele não sinta o pulso. Num led, rápida o bastante para que o olho humano não perceba piscar.
    - Motivo o uso é a eficiência pois normalmente se usa transistores para fazer a comutação e ele completamente ligado ou desligado dissipa menos calor que num estado intermediário.

Diferenças em código

- Escrita de dados analógicos
  - Pulse Width Modulation (PWM)



[4]

Diferenças em código

- Escrita de dados analógicos
  - Pulse Width Modulation (PWM)

```
#define PIN_LED 9
void setup(){
   pinMode(PIN_LED, OUTPUT);
}
void loop(){
   analogWrite(PIN_LED, 128); // 50% do ciclo
}
```

```
#define PIN_LED 5
int canal = 0;
int freq = 5000; // 5 kHZ
Int res = 8;         // 8 bits

void setup(){
   ledcAttach(PIN_LED, freq, res);
}
void loop(){
   ledcWrite(PIN_LED, 128);  // 50% do ciclo
}
```

Diferenças em código

- Escrita de dados analógicos
  - Estilo Arduino:
    - Utiliza timers internos do microcontrolador para gerar a modulação.
    - Frequência padrão: 490 Hz (em alguns pinos pode ser 980 Hz a depender do timer conectado ao pino).
      - Os timers 0 e 2 são de 8 bits e geram a frequência de 980 Hz, já o timer 1 usa 16 bits e gera a frequência 480 Hz. Alterar as configurações dos timers afeta outras funções que deles dependam (timer0 – millis, micros, delay).
    - Para gerar esse PWM no arduino (e ESP atualmente) usa-se:
      - analogWrite(pin, value) (arduino e ESP32).
      - analogWriteResolution(pin, resolution) (ESP32).
      - analogWriteFrequency(pin, freq) (ESP32).
    - Em placas com AVR como arduino UNO, a resolução do PWM é de 8 bits (de 0-255) para o value (ciclo).

Diferenças em código

- Escrita de dados analógicos
  - Estilo Arduino:
    - O ESP32 possui um periférico dedicado a geração do PWM, o LED Control, com as seguintes características:
      - 6-16 canais (depende do modelo).
      - Resolução de 1 até 20 bits (depende da frequência).
      - Frequência: 1 Hz a 40 MHz (depende da resolução).
        - Fórmula: $Freq_{max}$ = 80 MHz/(divisor (normalmente 1) x $2^{resolução}$).
    - O core do Arduino pode utilizar-se de funções mais avançadas que acessam diretamente esse hardware para gerar o PWM.
    - No caso do uso do ESP32 as funções são:
      - ledcAttach(pino, frequencia, resolucao).
      - ledcAttachChannel(pino, frequencia, resolucao, canal).
      - ledcWrite (pino, ciclo).
      - ledcWriteChannel(canal, ciclo).
      - ledDetach(pino).

Diferenças em código

- Escrita de dados analógicos
  - ESP-IDF:
    - No ESP-IDF o PWM é controlado pelo mesmo periférico LEDC, mas com diferenças.
    - O LEDC tem timers dedicados (4) no ESP32, por isso o programador precisa configurá-los para o canal que vai utilizar.
    - O programador também precisa configurar o canal (dos até 16) a utilizar e por fim o ciclo de trabalho.
    - Então no IDF os passos são:
      - Configurar um timer para o PWM (frequencia + resolução) usando o struct (ledc_timer_config_t) e função.
      - Configurar um canal para associar ao GPIO com o struct (ledc_channel_config_t) e função.
      - Ajustar o ciclo (duty) com o uso da função respectiva.

Diferenças em código

- Escrita de dados analógicos
  - ESP-IDF:
    - Funções:
      - ledc_timer_config(timer_config) – Configura o timer do ledc com o modo de velocidade, a fonte do timer, resolução e frequência.
      - ledc_channel_config(ledc_config) – Configura o canal do ledc com modo de velocidade, o canal, a fonte do timer, interrupção, gpio, valor do ciclo inicial.
      - ledc_set_duty(speed_mode, channel, duty) – Função que ajusta o canal para um novo ciclo.
      - ledc_update_duty(speed_mode, channel) – Função que atualiza os parâmetros do canal.
      - ledc_stop(speed_mode, channel, idle_level) – Função que desativa o ledc, ajustando sua saída para o valor idle_level.

Diferenças em código

- Escrita de dados analógicos
  - ESP-IDF:
    - Exemplo:

```
#include “driver/ledc.h”
void app_main(void){
   ledc_timer_config_t ledc_timer = {
      .speed_mode = LEDC_HIGH_SPEED_MODE,
      .duty_resolution = LEDC_TIMER_13_BIT,
      .timer_num = LEDC_TIMER_0,
      .freq_hz = 5000,
      .clk_cfg = LEDC_AUTO_CLK
   };
   ledc_timer_config(&ledc_timer);
```

```
   ledc_channel_config_t ledc_channel = {
      .speed_mode = LEDC_HIGH_SPEED_MODE,
      .channel = LEDC_CHANNEL_0,
      .timer_sel = LEDC_TIMER_0,
      .intr_type = LEDC_INTR_DISABLE,
      .gpio_num = 18,
      .duty = 0,
      .hpoint = 0
   };
   ledc_channel_config(&ledc_channel);

   ledc_set_duty(LEDC_HIGH_SPEED_MODE,
LEDC_CHANNEL_0, 4096); // 50% para 13 bits
   ledc_update_duty(LEDC_HIGH_SPEED_MODE,
LEDC_CHANNEL_0);
```

Diferenças em código

- Comunicação serial (UART – Universal Asynchronous Receive/Transmitter):
  - Para exibir dados no terminal (serial monitor) via comunicação serial, necessita-se habilitar esse meio de comunicação.
  - A UART transmite dados bit a bit, geralmente num formato 8N1:
    - 8 bits de dados.
    - Sem bit de paridade (N) – usado na detecção de erro de transmissão.
    - 1 stop bit (1).
  - Estilo Arduino:
    - Serial.begin(baudRate) – Função que define a velocidade de comunicação da serial. Se houver múltiplas UART, a função será Serial1, Serial2 e etc. Configuração dos pinos e buffers é realizada de forma automática.

Diferenças em código

- Comunicação serial (UART – Universal Asynchronous Receive/Transmitter):
  - Estilo Arduino:
    - Para enviar dados usa-se:
      - Serial.print(dado) – envia dados numa linha.
      - Serial.println(dado) – envia dados numa linha e ao final quebra para a próxima linha.
      - Serial.write(dado) – envia um byte bruto.
    - Para receber dados usa-se:
      - Serial.available() – verifica se tem dados no buffer de recepção.
      - Serial.read() – retorna o byte do buffer.
      - Serial.readBytes(buffer, length) – lê length bytes do buffer (com timeout caso não cheguem todos os bytes).
      - Serial.readBytesUntil(terminator, buffer, lenght) – lê da mesma forma que a anterior, mas com a possibilidade de parar se achar o terminator.

Diferenças em código

- Comunicação serial (UART – Universal Asynchronous Receive/Transmitter):
  - Estilo Arduino:
    - Para receber dados usa-se:
      - Serial.readString() – Retorna um tipo String com os bytes lidos da buffer, caindo no timeout se não houver mais bytes.
      - Serial.readStringUntil(caractereTerminador) – lê os bytes até achar o caractere terminador e retorna tudo como uma String.
  - Estilo ESP-IDF:
    - Necessita instalar o driver da UART
      - #include “driver/uart.h”
    - Necessita configurar os parâmetros da UART no seu struct (uart_config_t uart_config):
      - Baud_rate, data_bits, parity, stop_bits, flow_ctrl.

Diferenças em código

- Comunicação serial (UART – Universal Asynchronous Receive/Transmitter):
  - Estilo ESP-IDF:
    - Para aplicar a configuração, usa-se a função:
      - uart_param_config(UART_NUM_x, &uart_config);
    - Para mapear os pinos tx, rx, rts e cts para pinos específicos do ESP usa-se:
      - uart_set_pin(PORT_num, TX_IO, RX_IO, RTS_IO, CTS_IO).
      - Pode-se usar o UART_PIN_NO_CHANGE para manter as configurações anteriores.
    - Para instalar o driver da UART usa-se:
      - uart_driver_install(PORT_num, rx_buffer_size, tx_buffer_size, queue_size, handler_queue, flags_interrupção).
    - Para deletar um driver uart:
      - Uart_driver_delete(PORT_num).

Diferenças em código

- Comunicação serial (UART):
  - Exemplos:

```
Serial.begin(115200);
Serial.println("Olá, mundo!");
```

```
uart_config_t config = {
    .baud_rate = 115200,
    .data_bits = UART_DATA_8_BITS,
    .parity = UART_PARITY_DISABLE,
    .stop_bits = UART_STOP_BITS_1,
    .flow_ctrl = UART_HW_FLOWCTRL_DISABLE
};
uart_param_config(UART_NUM_1, &config);
uart_driver_install(UART_NUM_1, 1024, 0, 0, NULL, 0);
uart_write_bytes(UART_NUM_1, "Olá, mundo!\n", strlen("Olá, mundo!\n"));
```

Fontes

[1] – Autoria própria.
[2] – https://embarcados.com.br/pinos-digitais-do-arduino/.
[3] – https://blog.eletrogate.com/entendendo-os-resistores-de-pull-up-e-pull-down/.
[4] – https://www.upesy.com/blogs/tutorials/how-to-use-pwm-on-esp32-with-examples.

IMD0904 –Plataforma de Hardware para Internet das Coisas

Prof. Daniel Lopes Martins
daniel.martins@imd.ufrn.br

UFRN | metrópole DIGITAL